
Polynomial regression

Data representation



$$D = \{\mathbf{x}^{(i)}, f(\mathbf{x}^{(i)})\}$$

Univariate



Instance	Temperature (C)	Energy Consumption (kWh)
1	13.39	5.46
2	34.19	7.37
3	24.81	4.02

Multivariate



Instance	Temp (C)	Humidity	Pressure (ha)	Consumption (kWh)
1	13.39	0.81	1231	5.46
2	34.19	0.23	1638	7.37
3	24.81	0.45	1348	4.02

Recap: Multi-variate hypothesis



Univariate hypothesis: $h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x$

Multivariate hypothesis:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_m x_m$$

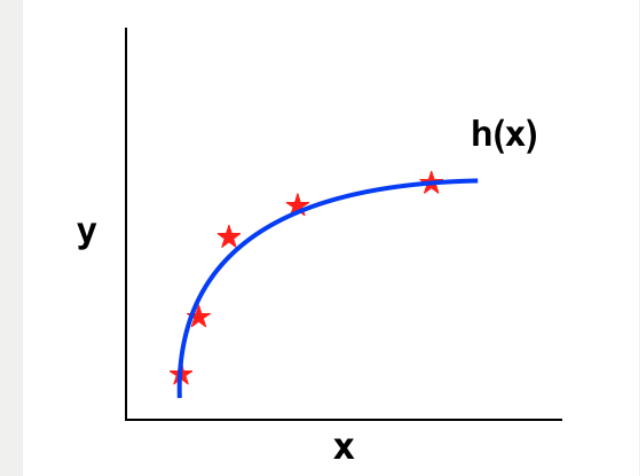
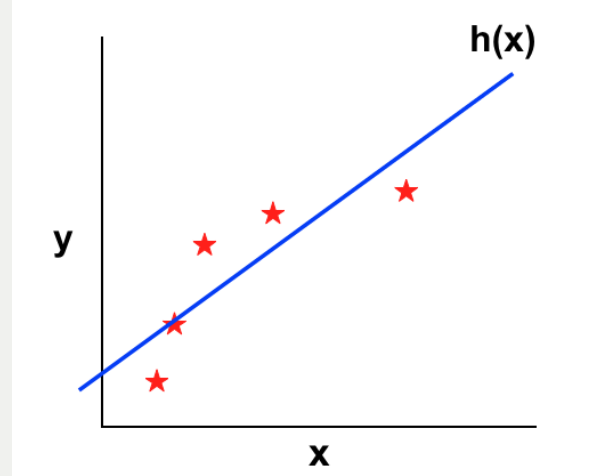
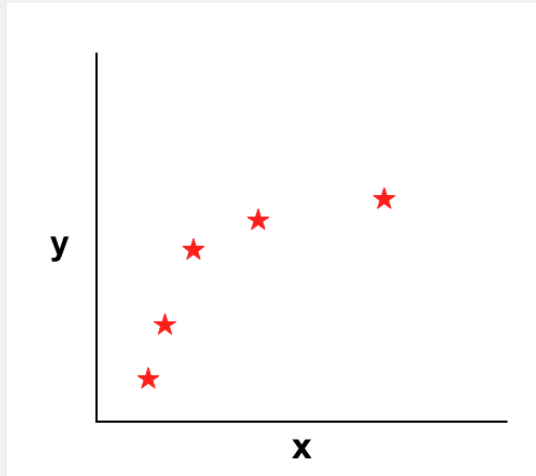
For m attributes (n is typically used for the size of the data set)

Polynomial regression



Linear regression model is too simple and does not have enough **capacity** to represent the data.

How can we increase capacity of the model?



Polynomial regression



Assumes that the target function (learning task) is:
A polynomial equation

Hypothesis:

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \dots + \theta_i x_m + \theta_j x_m^2 + \dots$$

Add higher order terms to the hypothesis.

Polynomial regression



Polynomial regression is implemented by using a simple trick.

- Change the features to polynomial features
- Fit linear regression model to new features

e.g., Polynomial feature of degree 2 for our univariate case:

$$h(\mathbf{x}) = \theta_0 + \theta_1 x_1$$

Instance	x	y
1	13.39	5.46
2	34.19	7.37
3	24.81	4.02

$$h(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2$$

Instance	x	x^2	y
1	13.39	179.3	5.46
2	34.19	1168.9	7.37
3	24.81	615.5	4.02

Polynomial regression



The loss function is the same:

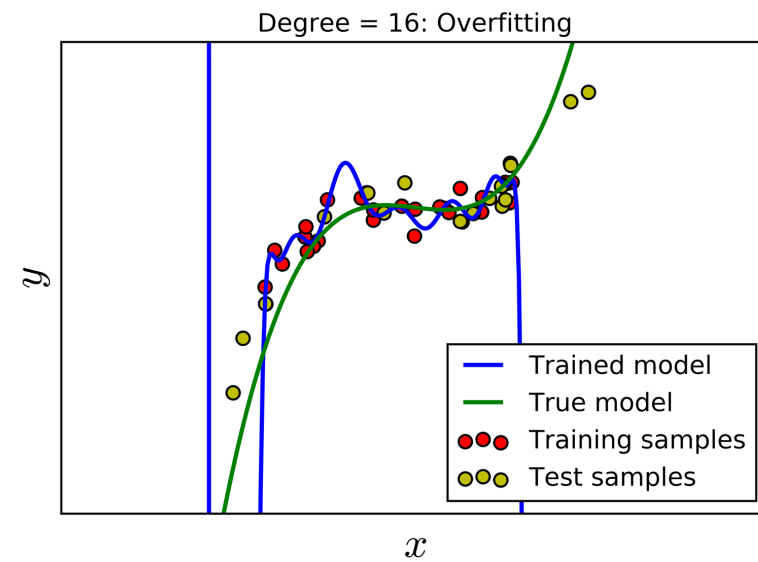
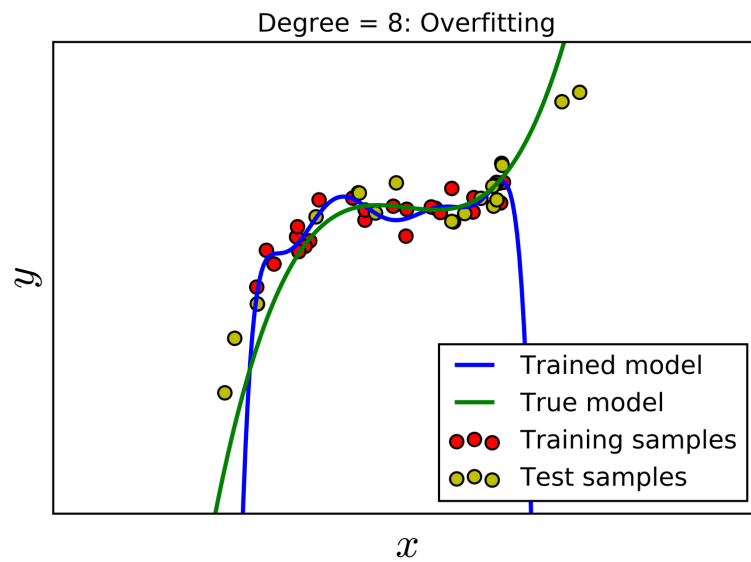
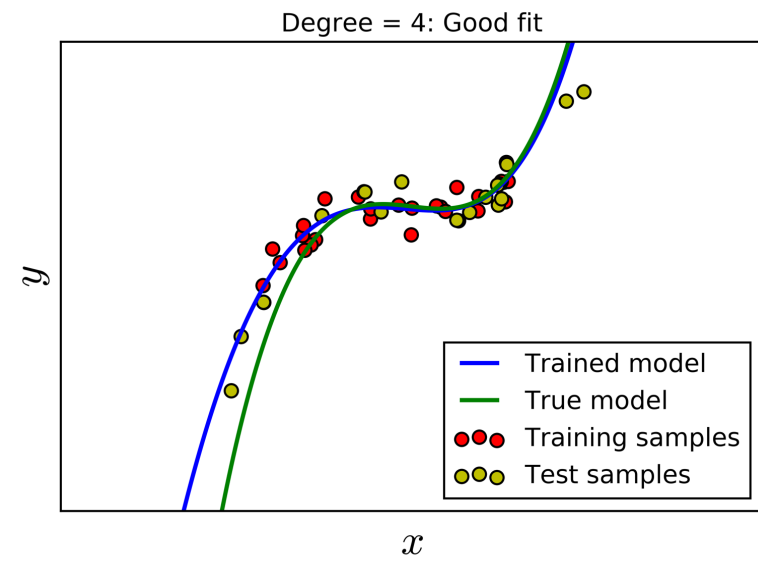
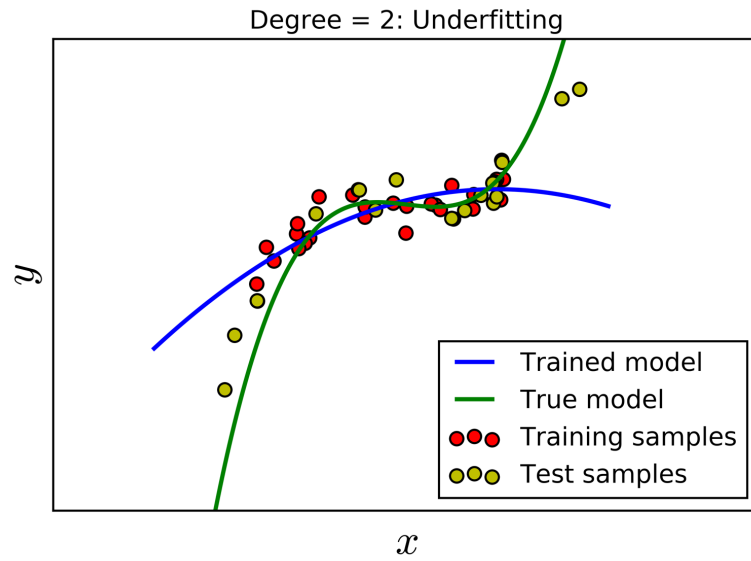
$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

In theory, the same gradient descent approach can be used

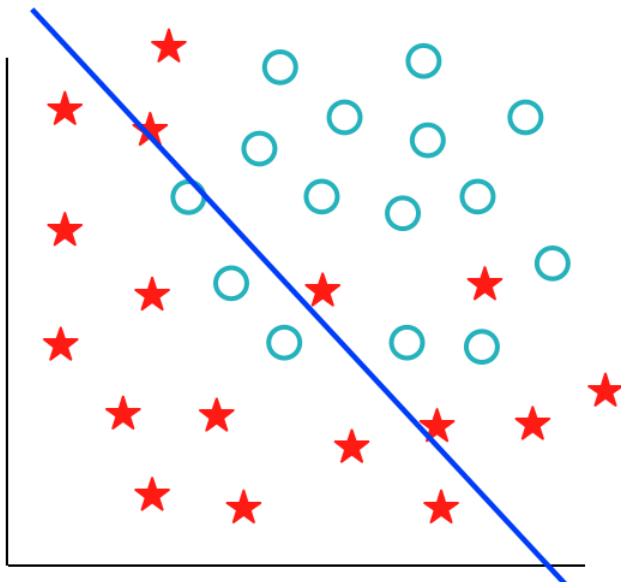
However,

Feature normalization becomes very important.

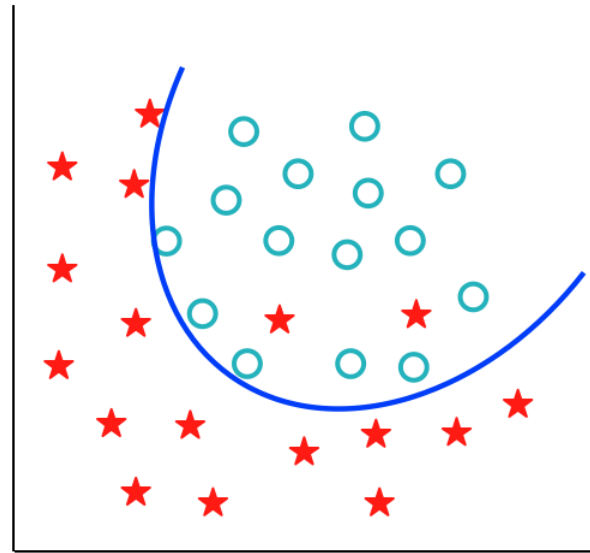
Regularization



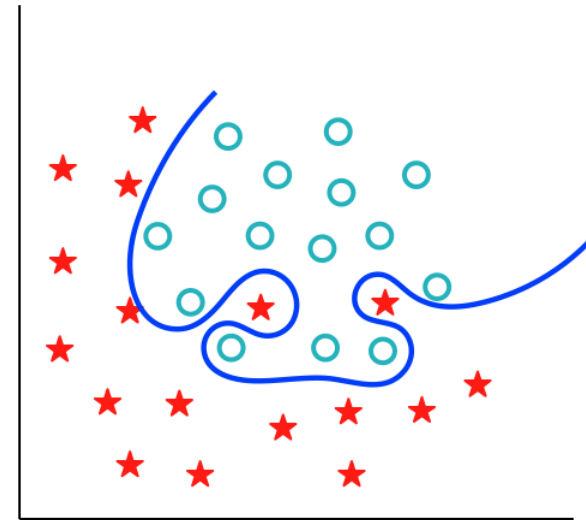
Regularization – Intuition



???

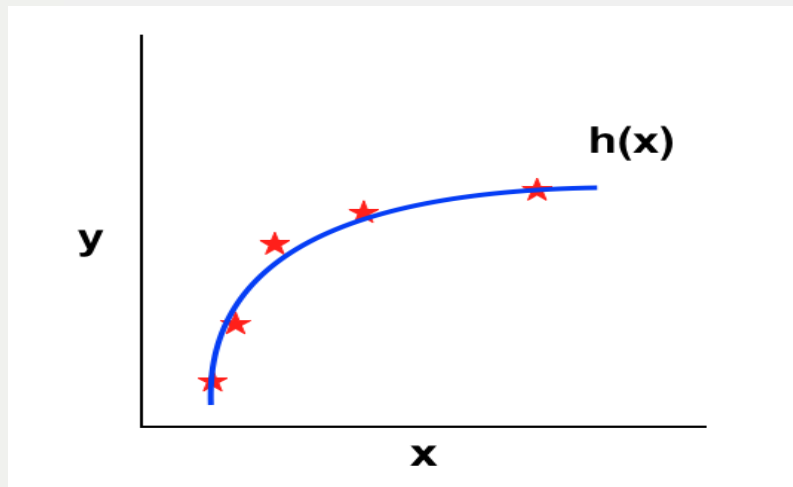


???

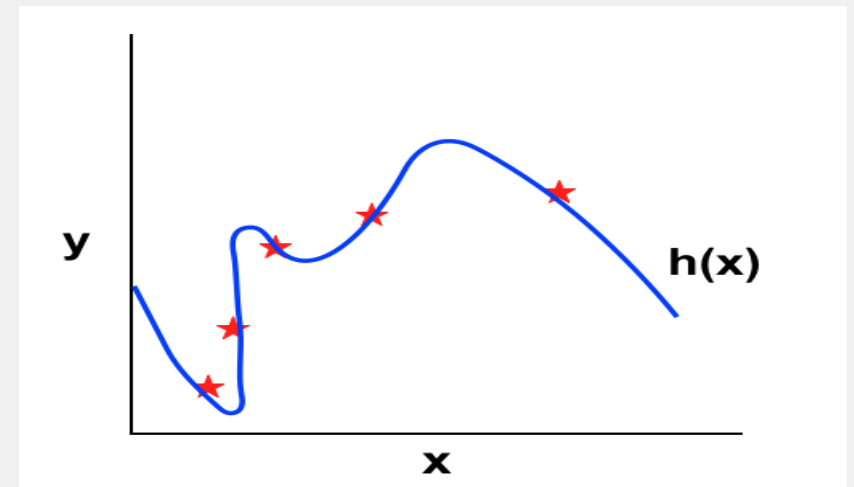


???

Regularization – Intuition



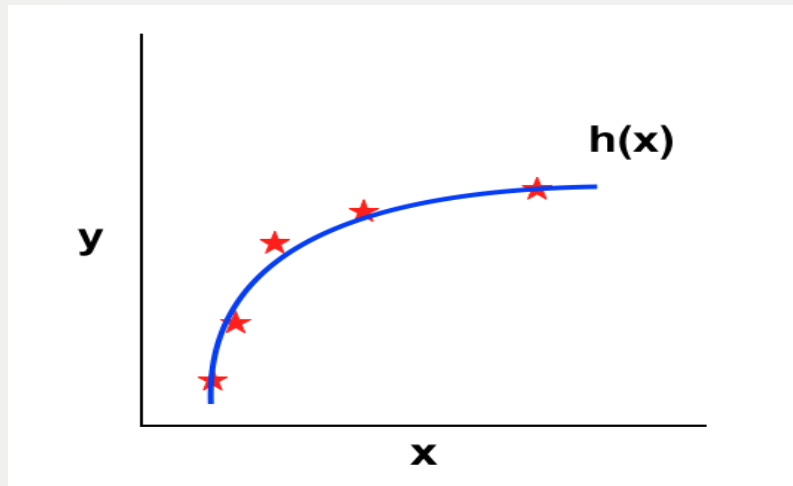
$$\theta_0 + \theta_1 x_1 + \theta_2 x_1^2$$



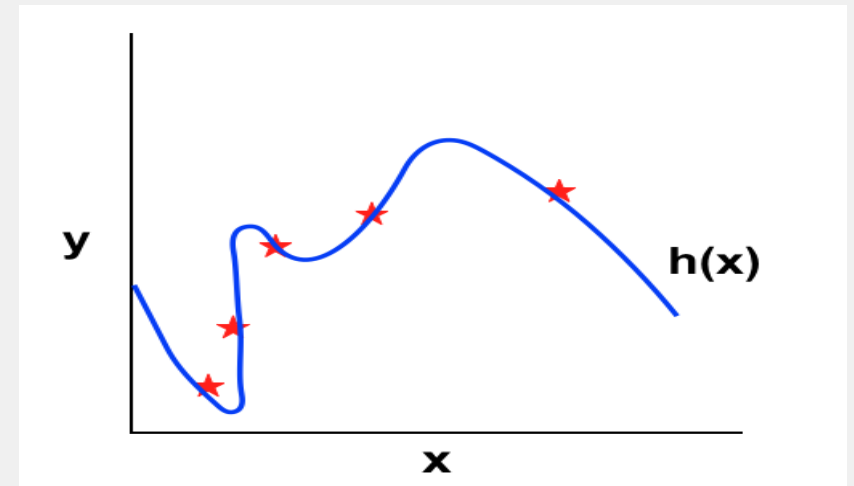
$$\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_1^4$$

How to **convert high degree to low degree** without changing the form/shape of the hypothesis?

Regularization – Intuition



$$\theta_0 + \theta_1 x_1 + \theta_2 x_1^2$$

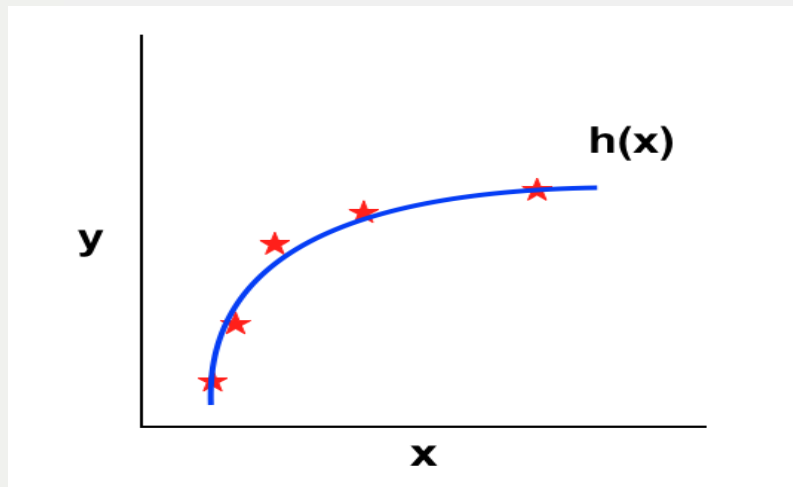


$$\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_1^4$$

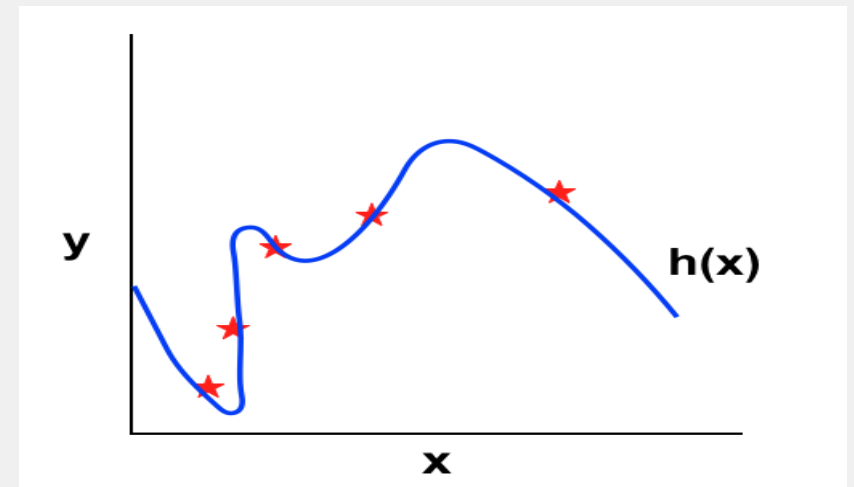
Constrain/regularize or **shrink the coefficient estimates towards zero**.

Discourage learning a more complex or flexible model, to avoid the risk of overfitting.

Regularization – Intuition



$$\theta_0 + \theta_1 x_1 + \theta_2 x_1^2$$



$$\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_1^4$$

Penalise θ_3 and θ_4 to make it very small

That is, we don't have to "decide" which form of the hypothesis to use

Regularization

Regularization is the concept of “regulating” the weights

Find the parameters that “matter the most”

Encourage small values for those parameters

Find a **simpler hypothesis**

- For polynomial regression, generally lower order
- Lower weights “across-the-board”

Less prone to **overfitting**

Regularization of the loss function



Introduce the **regularization term** to the loss-function

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{v=1}^m \theta_v^2$$

Still find: $\min_{\theta} J(\theta)$

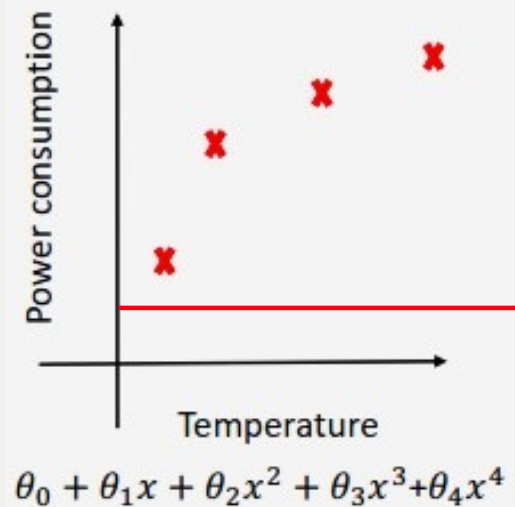
Effect of λ (regularization weight)



$$J(\theta) = \min_{\theta} \left[\frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{v=0}^m \theta_v^2 \right]$$

What if λ is set to an extremely **large** value?

(perhaps far too large for our problem, say $\lambda = 10^{100}$)



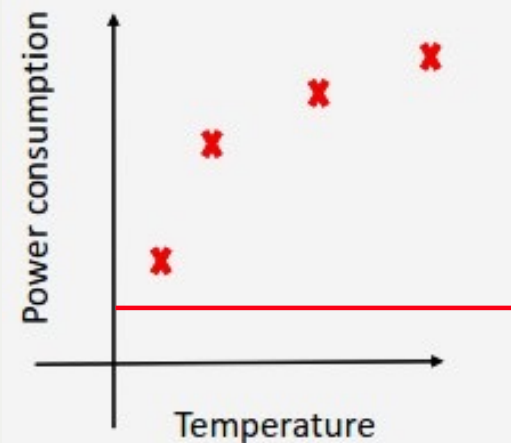
Effect of λ (regularization weight)



$$J(\theta) = \min_{\theta} \left[\frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{v=0}^m \theta_v^2 \right]$$

What if λ is set to an extremely **large** value?

(perhaps far too large for our problem, say $\lambda = 10^{100}$)



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4 \quad \theta_1, \dots, \theta_4 \rightarrow 0$$

- Drives all θ to zero
- Algorithm likely to **under-fit**
(fails to fit even training data well)

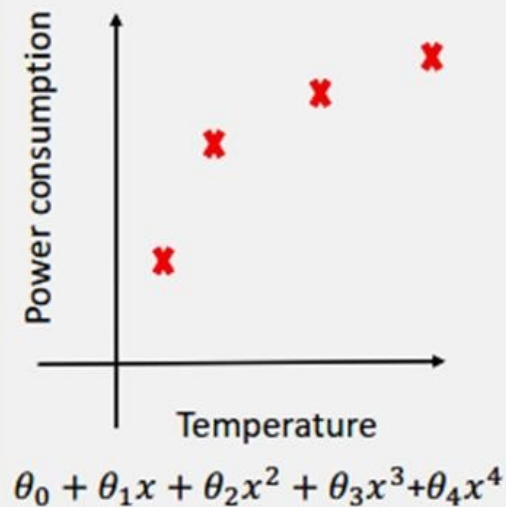
Effect of λ (regularization weight)



$$J(\theta) = \min_{\theta} \left[\frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{v=0}^m \theta_v^2 \right]$$

What if λ is set to an extremely **small** value

(perhaps far too large for our problem, say $\lambda = 10^{-100}$)



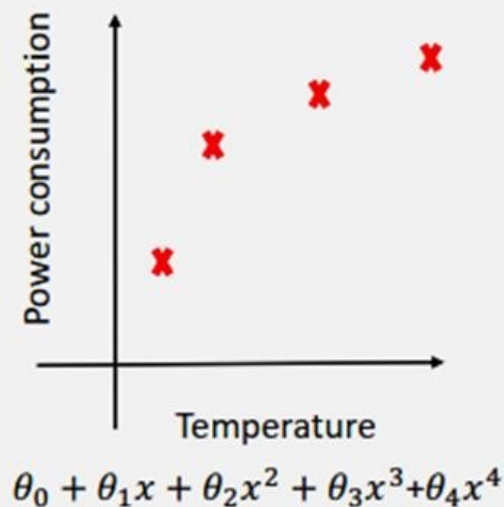
Effect of λ (regularization weight)



$$J(\theta) = \min_{\theta} \left[\frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{v=0}^m \theta_v^2 \right]$$

What if λ is set to an extremely **small** value

(perhaps far too large for our problem, say $\lambda = 10^{-100}$)



- Regularization has almost no effect
- Algorithm likely to **over-fit** (fits training data well)

Setting the regularization term

λ relates to the **complexity of the model**

Large λ , lots of regularisation, less complex model
(possibly under-fit)

Small λ , little regularisation, more complex model
(possibly over-fit)

Alternative methods



There are many methods for regularization

The one presented in this lecture is known as **Ridge Regression (L2)**

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{v=1}^m \theta_v^2$$

This is also what it is called in the sci-kit learn tools

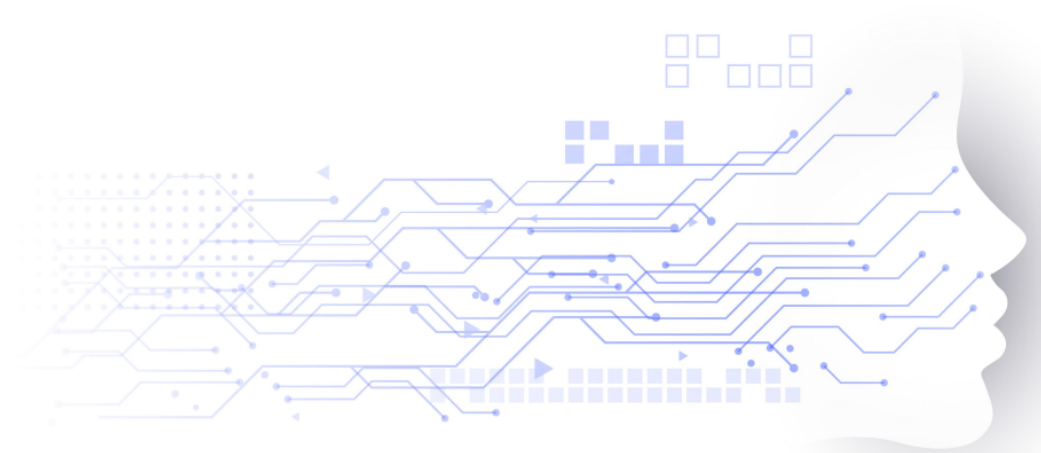
Another very popular method is **Lasso Regression (L1)**

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{v=1}^m |\theta_v|$$

Alternative methods

The main differences:

- In ridge regression, all weights will be non zero, but many will be very small.
- In lasso regression, some weights may be zero, effectively conducting **feature selection**

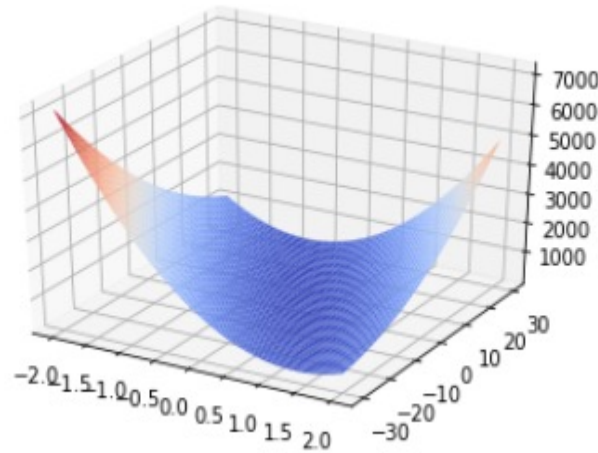


Logistic Regression

COSC2753 Machine Learning

Semester 1, 2026

Recap: Regression: a form of supervised learning



Experience: $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^n$ $\mathbf{x} \in \mathbb{R}^m$ $y \in \mathbb{R}$

Hypothesis space:

- **Linear:** $h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m = \theta^{\top} \mathbf{x}$
- **Polynomial:**
$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \dots + \theta_i x_m + \theta_j x_m^2 + \dots$$

Loss function: $J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$

Optimization: Gradient decent

Repeat $\left\{ \begin{array}{l} \theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad \forall j \\ \end{array} \right.$

Classification

A type of supervised learning

Classification

Instead of output are continuous/real valued,
now they are **discrete labels or classes**

For example

Image recognition: cat/dog

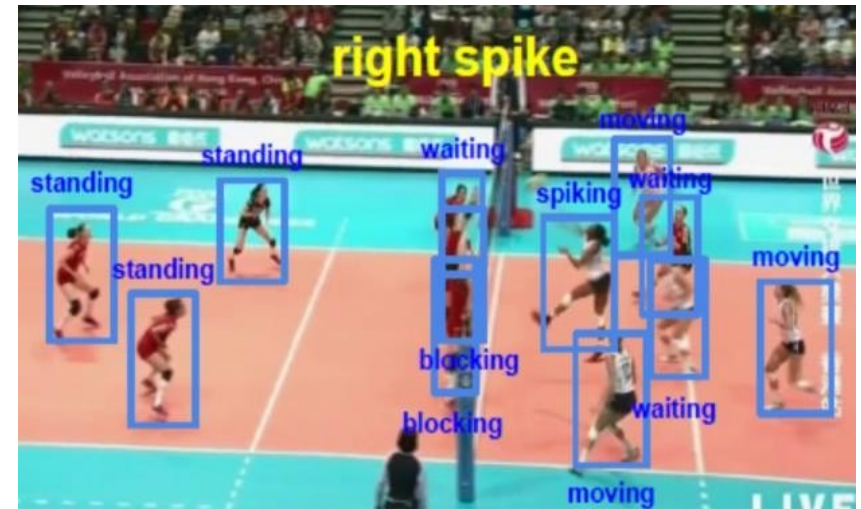
Sentiment recognition:
positive/neutral/negative

Credit card transaction: fraudulent/legitimate

Tumour: benign/malignant

Binary class: $y \in \{0, 1\}$, $y \in \{+1, -1\}$

Tertiary (or n-ary) class: $y \in \{0, 1, 2, \dots, k\}$ or $y \in \{cat, dog, \dots\}$



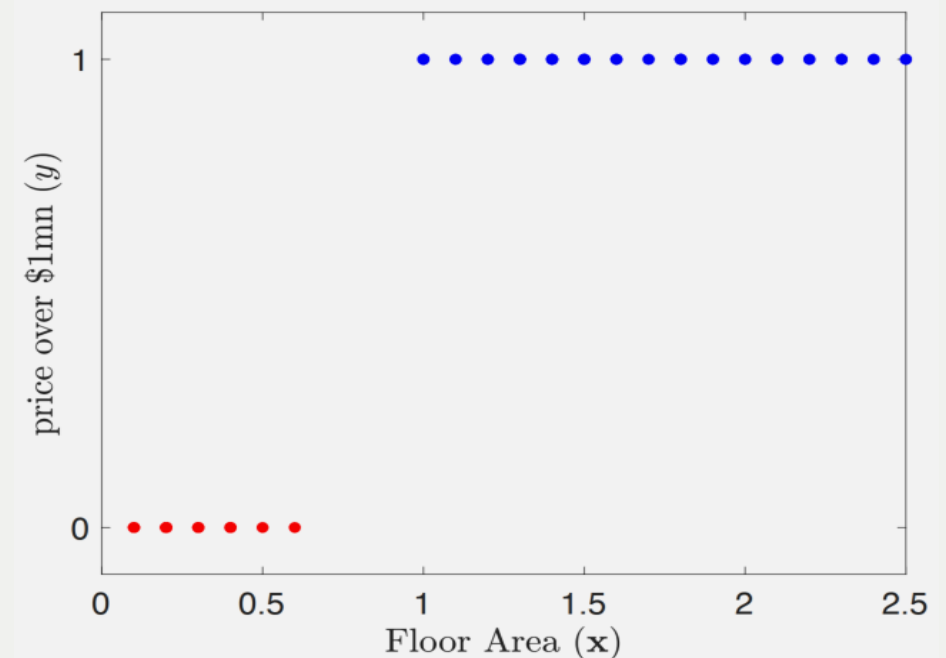
Classification



Consider **univariate** and binary class case

In a continuous space, for classification what is:

- The input?
- The output?



Regression vs Binary classification

Regression:

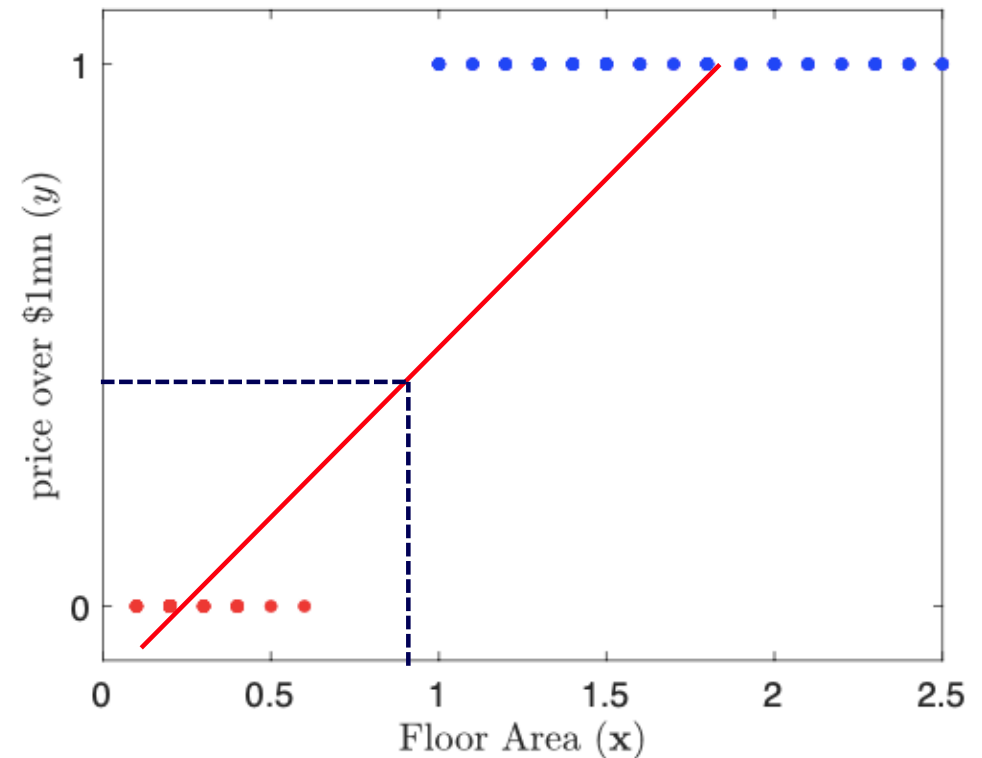
Probability of the fact “price over \$1m”



Binary classification:

threshold output $h_{\theta}(\mathbf{x})$

- If $h_{\theta}(\mathbf{x}) \geq 0.5$, predict $y = 1$
- If $h_{\theta}(\mathbf{x}) < 0.5$, predict $y = 0$

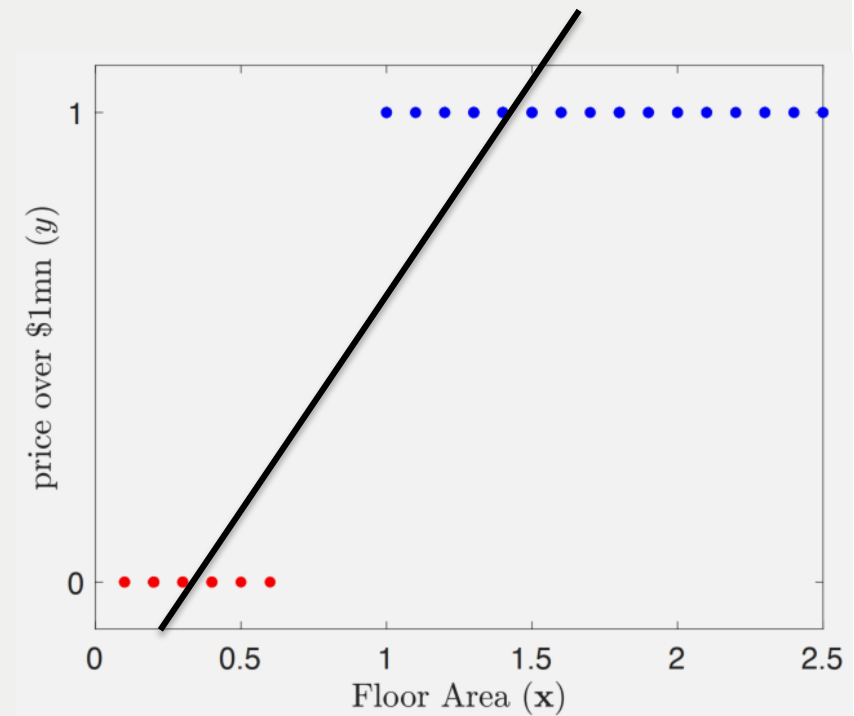


Linear Regression vs Binary classification



Issues :

- But, $h_{\theta}(\mathbf{x})$ can be larger than 1 or less than 0
- Regression line doesn't fit the classification problem very well



Hypothesis Space

for logistic regression

Sigmoid function

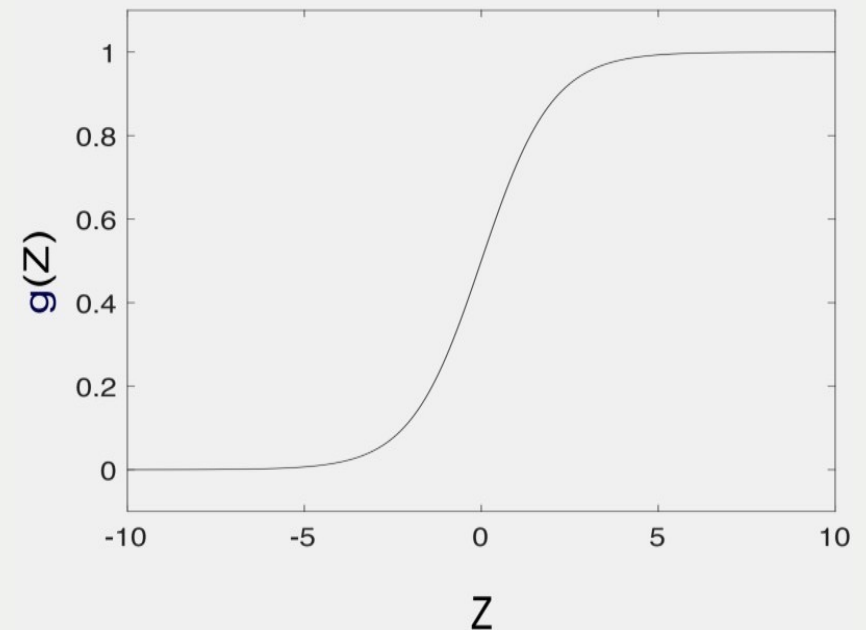


The **sigmoid function** or **logistic function**

$$g(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- Between 0 and 1
- Continuous and smooth
 - Can take derivatives



Logistic regression hypothesis

Goal: perform **binary classification** based on linear model of attributes

$$0 \leq h_{\theta}(\mathbf{x}) \leq 1$$

Logistic Regression:

$$h_{\theta}(\mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_m x_m) = g(\theta^T \mathbf{x})$$

Function $g(z)$ is the sigmoid function

$$g(z) = \frac{1}{1 + e^{-z}}$$

Importantly, we have $0 \leq h_{\theta}(\mathbf{x}) \leq 1$ for all inputs



Logistic regression hypothesis

Use the **sigmoid function** to transform our linear hypothesis into the logistic hypothesis.

Hence for Logistic Regression:

$$h_{\theta}(\mathbf{x}) = g(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

Importantly, we have $0 \leq h_{\theta}(\mathbf{x}) \leq 1$ for all inputs

However, what actually is $h_{\theta}(\mathbf{x})$?

Interpretation of hypothesis



$h_{\theta}(\mathbf{x})$ estimates the probability that $y = 1$ for input x

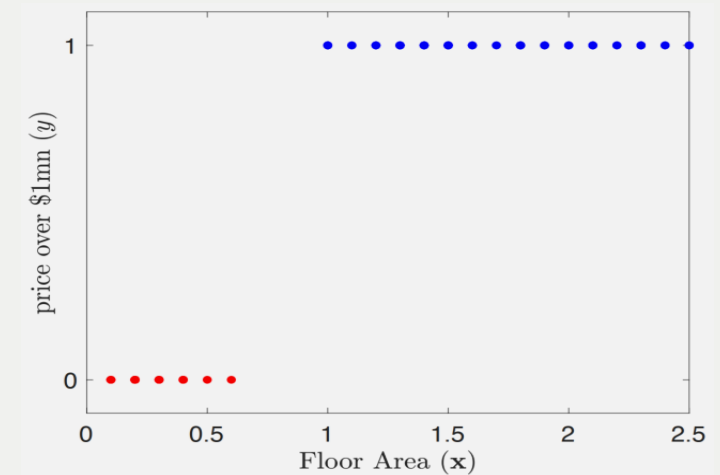
Mathematically:

$$h_{\theta}(\mathbf{x}^{(i)}) = P(y^{(i)} = 1 | \mathbf{x}^{(i)}, \theta)$$

It is NOT the actual value of “ y ”

For example, if $x = 0.9$ then $h_{\theta}(\mathbf{x}) = 0.75$

➔ a house with floor area "0.9" units, has a 75% chance of having the price over \$1mn.



Notation

Conditional Probability notation:

$$P (A = x | B = y)$$

The probability that random variable A takes value x,
given that random variable B has taken value y.

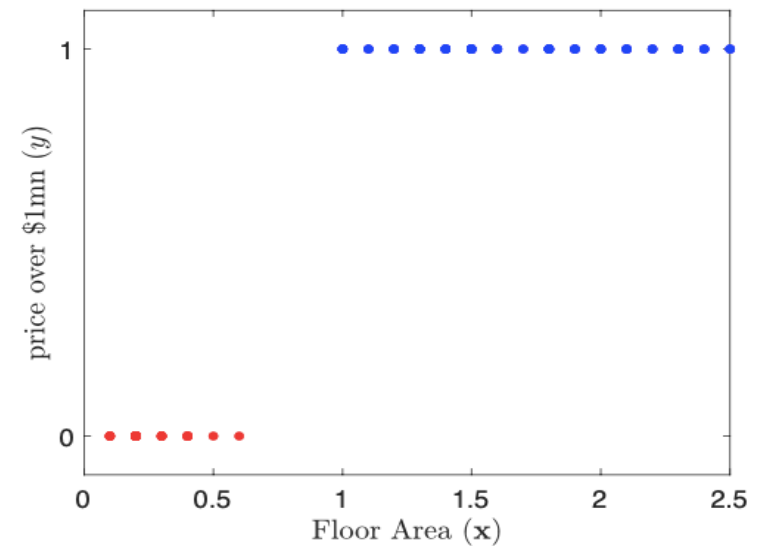
Interpretation of hypothesis

To get probability of $y = 0$, recall:

$$P(y = 0|\mathbf{x}, \theta) + P(y = 1|\mathbf{x}, \theta) = 1$$

Therefore,

$$\begin{aligned} P(y = 0|\mathbf{x}, \theta) &= 1 - P(y = 1|\mathbf{x}, \theta) \\ &= 1 - h_{\theta}(\mathbf{x}) \end{aligned}$$



Decision boundary

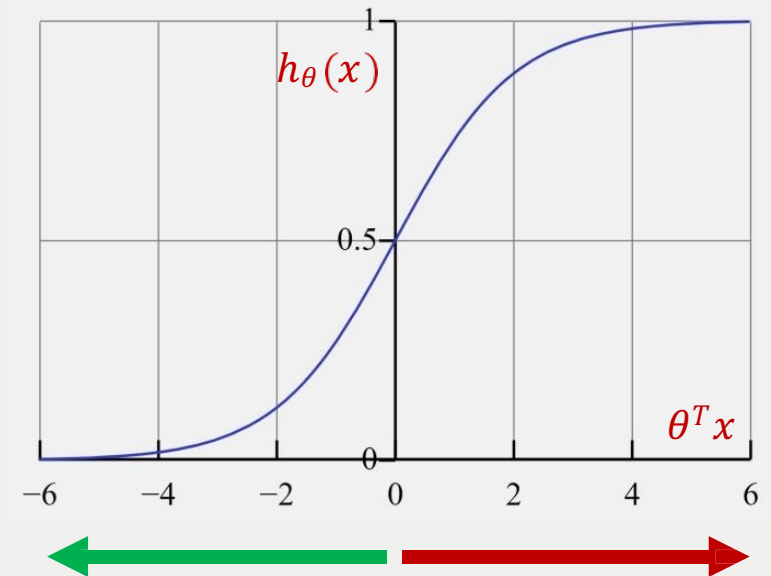
The decision boundary is the point at which the binary classification changes:

$$h_{\theta}(\mathbf{x}) = g(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

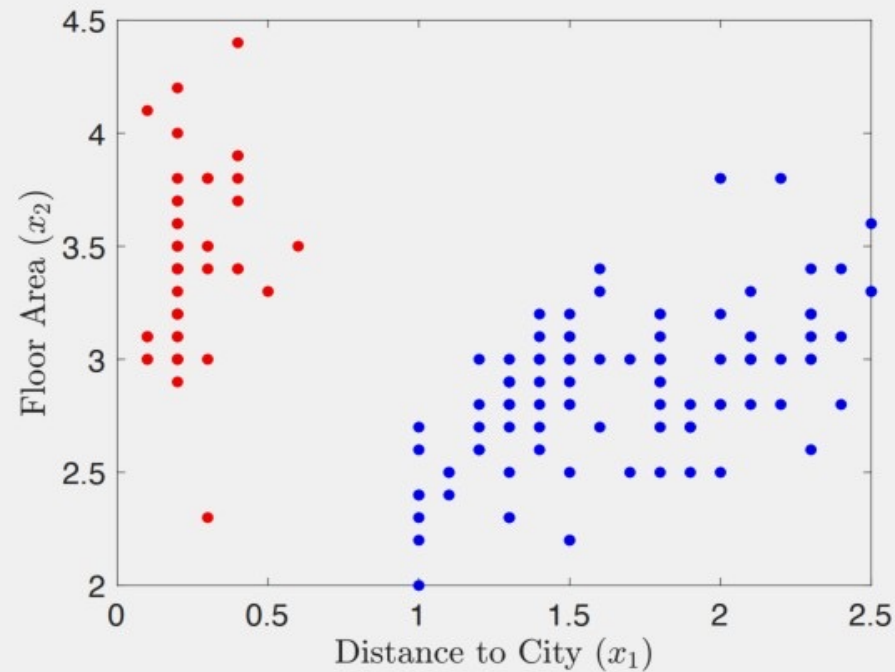
Predict $y = 1$ if $h_{\theta}(\mathbf{x}) \geq 0.5$
 $\theta^T \mathbf{x} \geq 0$

Predict $y = 0$ if $h_{\theta}(\mathbf{x}) < 0.5$
 $\theta^T \mathbf{x} < 0$

Observe that **only need to consult the weights**, not the sigmoid!



Decision boundary



$$h_{\theta}(\mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

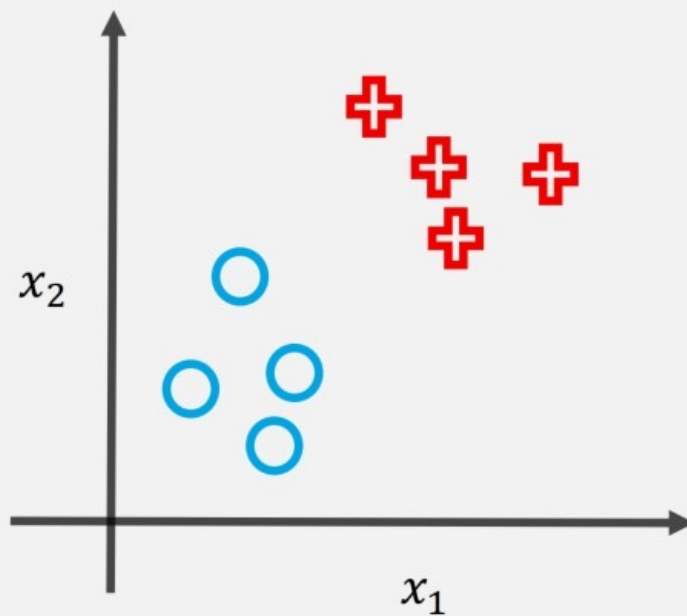
Predict $y = 1$ if $\theta_0 + \theta_1 x_1 + \theta_2 x_2 \geq 0$

Multi-class classification

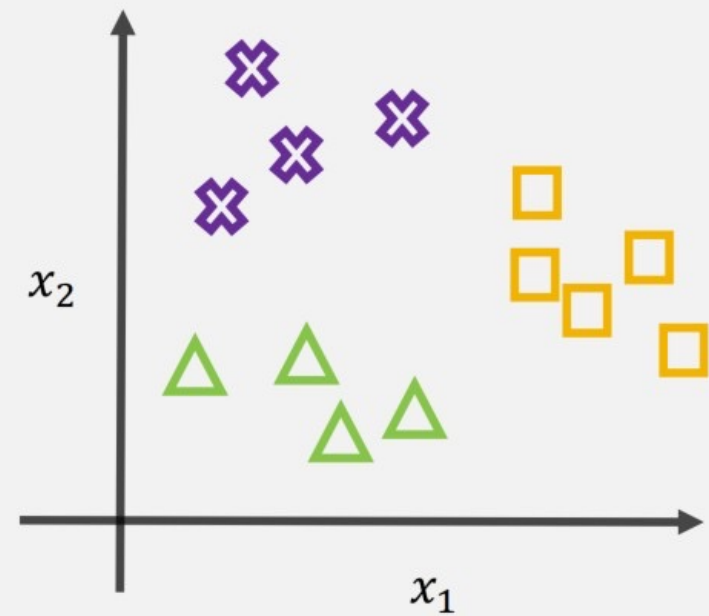
Binary vs Multi-class Classification



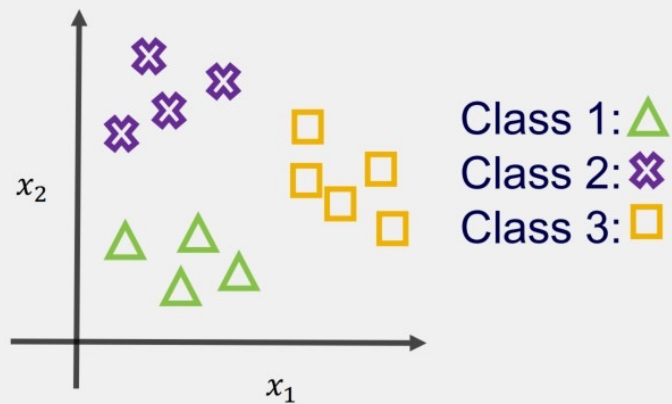
Binary Classification



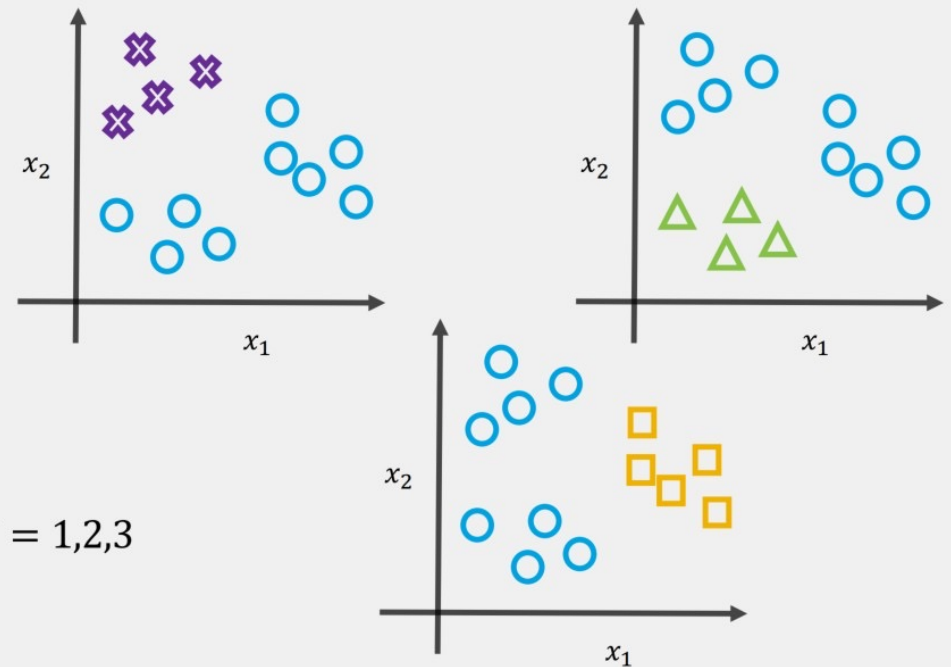
Multi-class Classification



One-versus-all



$$h_{\theta}^j(\mathbf{x}) = P(y = j | \mathbf{x}, \theta) \forall j = 1, 2, 3$$



One-versus-all

Train a logistic regression classifier $h^j(\mathbf{x}_\theta)$
for each class j
to predict the probability that $y = j$

Given a new input $\mathbf{x}^{(*)}$ with unknown class, we predict its class j to be:

$$\max_j h_\theta^j(\mathbf{x}^{(*)})$$

Revision

- Polynomial & Regularization
 - Avoid overfitting
 - Encourage small values for parameters $\theta_0, \theta_1, \dots, \theta_m$
 - Find a simpler hypothesis
- Logistic regression
 - Sigmoid function: smooth and result in (0,1).
 - Can be used for classification
- Multi-class classification



Postpone Tutorial w4 – Wed 15-07-2026

- Make up for Tutorial w4 Thursday 15-07-2026
 - Room: 1.3.006
1. **Option 1:** 14h30-16h00 Friday (17-07-2026) – Room 1.3.006
 2. **Option 2:** 10h30-12h00 Friday (17-07-2026) – Online

Microsoft Teams meeting

<https://teams.microsoft.com/meet/41463777287811?p=hjx4RuT9nzYLMue8KL>

Meeting ID: 414 637 772 878 11

Passcode: bb9CY6jZ

3. **Option 3:** 10h30-12h00 Saturday (18-07-2026) – Room 1.3.006



Thank you!